

DESIGN OF A LOW POWER NETWORK INTERFACE FOR NETWORK ON CHIP

K. Swaminathan^{1,2}, G. Lakshminarayanan², Frank Lang³, Maher Fahmi³ Seok-Bum Ko¹
¹Department of Electrical and Computer Engineering, University of Saskatchewan, Canada.

²Department of ECE, National Institute of Technology, Tiruchirapalli, India.

³PMC-Sierra Inc, Canada.

{swami.nitt@gmail.com, laksh@nitt.edu, frank.lang, maher.fahmi@pmcs.com seokbum.ko@usask.ca}

ABSTRACT

In this paper, a low power flexible Network Interface (NI) Architecture for Network on chip (NoC) is proposed. The flexible run time configuration controller in the proposed NI plays a vital role to reduce the power by enabling and disabling the entire asynchronous First In First Outs (FIFOs) based on the traffic conditions between the router and the processing elements (PE). The NI has been implemented in Xilinx Virtex-5 XC5VLX110T FPGA. Experimental results show that the proposed low power NI offers a power improvement of 37%, when both FIFOs are inactive and 32 %, when only one FIFO is active.

Index Terms— Network on Chip, Network Interface, Asynchronous FIFO

1. INTRODUCTION

Increasing number of reusable IP cores in a Multi-Processor System on Chip (MPSoC) design leads to design and testing challenges. The NoC architecture is emerging as a promising solution for highly scalable, reliable and modular on-chip communication infrastructure platform which shifts the design methodologies from computation-based to communication-based [1-2]. The NoC architecture uses layered protocols which consist of on-chip routers, links and NIs on a predefined topology. It is essential to develop a plug and play generic NI to transfer the data between the router and the PEs. Also we need to have a mechanism to handle clock domain crossing issues in order to pass the data between different clock domains without loss [3].

In this paper, a low power micro architecture of WISHBONE bus compatible NI with the flexible configuration controller is proposed. It enables short design time and offers seamless high throughput data flow and also enables the users to connect various IPs with the routers even though the frequencies of the modules are different. The proposed generic NI offers a simple and flexible connection mechanism utilized to connect a single processing core to the router directly as well as multiple processing cores with processor, memories and peripherals connected through other standard on-chip buses to the router. The flexible configuration controller saves the power

by disabling the asynchronous FIFO based on the traffic requirements.

2. RELATED WORK

Several studies have been explored in the NI implementation for NoC to overcome the asynchronous problem between clock domains, standardization of NI fabric, speed and throughput improvement. Generally the implementations are based Direct Memory Access with asynchronous FIFO (DMA) [4], Globally Asynchronous Locally Synchronous (GALS) [6, 7], Advanced Microcontroller Bus Architecture (AMBA) [8] and AMBA Advanced eXtensible Interface (AXI) bus [9] Open Core Protocol (OCP) [10-11].

In [8], the network architecture exploits AXI transaction based protocol to be compatible with existing IP cores utilizing shared-memory abstraction and flexible instantiation on runtime using credit based handshaking protocol with NI kernels and NI Shells are proposed. In [9] the author exploits NI with improved memory utilization, less memory latency, packet reordering mechanism to handle the in-order delivery with the advance extensible interface to maintain compatibility with the existing intellectual property cores. In [12], the authors proposed a generic high-speed NI for NI using Ping Pong Buffers which utilizes four memory modules to increase the throughput.

In Section 3, essential requirements of NI for NoC and packet format of NoC used in this proposed design are explained. Section 4 describes the salient features of asynchronous FIFO used in the NI. Section 5 describes the architecture of proposed flexible configurable WISHBONE compatible NI. Section 6 explains about the performance of WISHBONE compatible NI under different scenarios. Finally Section 7 deals with conclusion and future work.

3. NI REQUIREMENT FOR NOC

3.1. Network Interface – An Overview

The NoC is composed of routers, NIs and links. NoC router/switch transports the data from incoming ports to outgoing ports to connect the adjacent router or NI. The NI

module connects the processing element to the NoC router. NIs convert request and response transaction into packets and vice versa [13]. The success of the NoC design is based on the standardization of the interfaces between IP cores and the routers. The general structure of NI shown in Figure 1 will have the following functions [13]:

1. Writing/reading the flits to/from the processing core and the router and vice versa.
2. Packing the incoming signals from the IP cores by assembling the number of flits, header information insertion, flit type insertion with exact number of flits and unpacking the signals coming from the router as per the IP core specification.

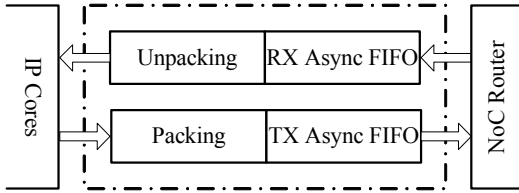


Figure 1. General Block diagram of NI for NoC

3. Transferring data from one clock domain to another clock domain without loss.

3.1. NoC Packet format

In order to improve the efficiency of the resource allocation in NoC, the message is split into packets for the allocation of control state and flow control digits (flits) for the allocation of channel bandwidth and buffer capacity [1]. The packets in the proposed work consist of flit with 40-bit width. The header flit is located at the first position of the packet contains information about the source and destination address, size of the packet and any other application specific commands. The packet format presented in [12] is utilized in the proposed design.

4. FIFO AND WISHBONE BUS FEATURES

4.1. Asynchronous FIFO

Asynchronous FIFOs are used to transfer the data from one clock domain to another clock domain without any loss in the data [5]. This requires a memory architecture which has two memory ports, one for input (or write or push) operation and another for output (or read or pop) operation. FIFO pointers are used to keep track of the read and write locations and also prevent the overflow and underflow of the FIFO buffer. FIFOs have the inherent characteristic of synchronizing itself to the read and write pointers.

The FIFO is said to be full when the read pointer catches up with the write pointer and the FIFO is said to be empty when the write pointer catches up with the read pointer [5]. Pointers must be one bit larger than the bits needed to address the FIFO memory. The comparing unit of

asynchronous checks the equality of the read and write pointers excluding the MSB bit. If both the pointers are equal then the FIFO must be either empty or full. If they are equal then it checks for the MSBs of the pointers. If both are equal, then the pointers have wrapped up the same number of times then the FIFO is empty. If both the MSBs are not equal then the FIFO is full [5].

4.2. Opencores WISHBONE Bus

The royalty free WISHBONE bus architecture offers a flexible design methodology to interconnect semiconductor IP cores. The main purpose is to foster design reuse by alleviating SoC integration problems. This is accomplished by creating a common interface between the IP cores. This improves the portability and reliability of the system, and hence results in faster time-to-market for the end user [4].

5. WORKING OF FLEXIBLE NI FOR NOC

The detailed architecture diagram of Flexible NI for NoC is shown in Figure 2. The NI consists of Master/Slave wrapper module, packing, unpacking, configuration controller and two asynchronous FIFO Modules.

5.1. WISHBONE Master and Slave

The Master/Slave interface module at the processing core side generates the required signals to write the data into the FIFO/packing unit and read out the data from the unpacking module and generates the required signals to translate the unpacking/packing module signals to WISHBONE compatible signals and generates the necessary control signals to write/read the data from NoC router to FIFO. Similarly, the Master/Slave interface module at the processing core side generates the required signals to write the data into the asynchronous FIFO via configuration controller and readout the unpacked data from the unpacking module and generates the required signals to translate the unpacking module signals to WISHBONE compatible and generates the necessary control signals to read from the asynchronous FIFO data to processing core. This interface performs single/block read/write operations on both sides. The WISHBONE reset (RST_I) signal is active LOW and is used to reset the IPs and the bus.

5.1.1. Processing core to NI Wrapper

During the reset state, the ACK_O signal is asserted LOW with respect to IP clock. A write operation is performed when WE_I is HIGH and FIFO is not FULL. This asserts ACK_O to HIGH which indicates a write transfer from the external Master to internal Slave write is permissible, and then the data bus (DAT_I) directly connected to the FIFO writes data when CYC_I, STB_I signal is HIGH as well as the FIFO Write signal is asserted HIGH. The address signal is not necessary when interfacing FIFO with WISHBONE

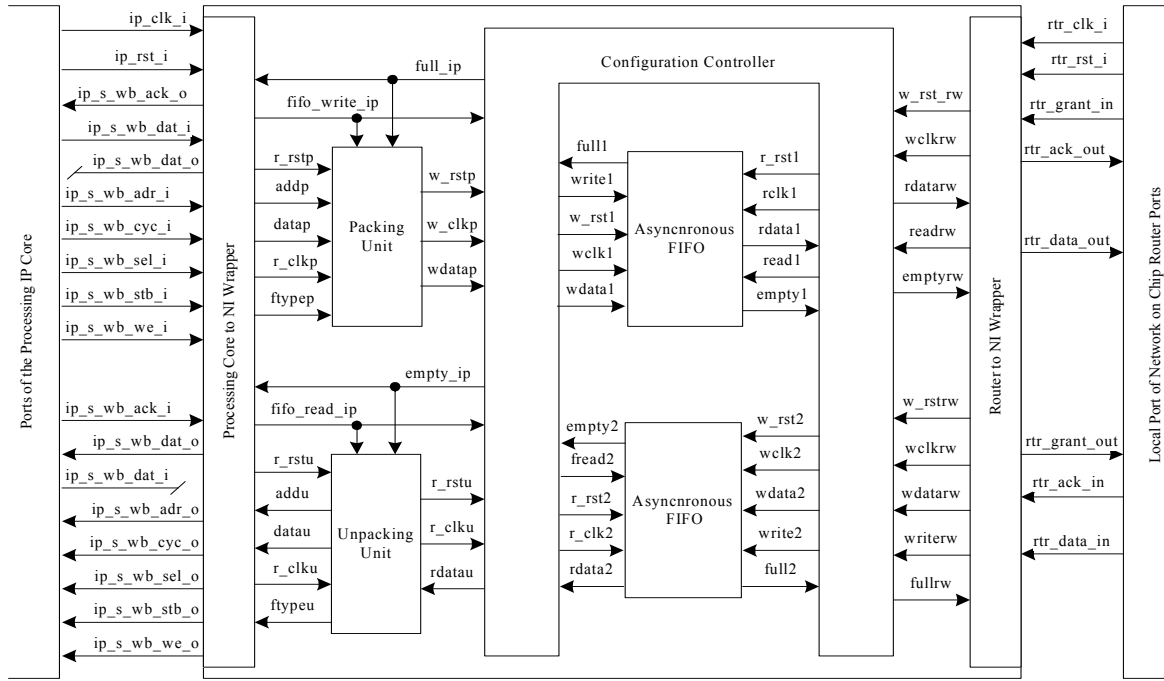


Figure 2. Flexible Network Interface Architecture

bus [5]. A write operation is not allowed when WE_I and FIFO's FULL flag is asserted HIGH. This indicates that the writing of data is impossible when the FIFO is full. The DAT_O and ADD_I are not used during this writing operation. The read operation on asynchronous FIFO via unpacking module by WISHBONE slave is identical with write operation except WE_O signal is asserted LOW and the read operation is permissible when FIFO is not empty.

5.2. Packing and Unpacking Module

The packing and unpacking modules involve header phrasing, assembling and disassembling of incoming and outgoing data as per the router, processing core frequency, packet size and data width.

5.2.1. Packing module

The packing unit receives data from WISHBONE master and packs the data with the necessary information of an individual packet [4], from the incoming flits and transfer to the asynchronous FIFO. The packing module forms the packet header with necessary routing information, number of flits, sequence number and control information required by the router and the IP core.

5.2.2. Unpacking module

The unpacking unit receives data from asynchronous FIFO via configuration controller and extracts the header information such as address, flit type, sequence number and packet size. Based on the packet size and control information, the data transfer is carried out by the respective modules in the local IP core through the WISHBONE slave interface. This module generates the local address of flits and flit type. The local address is directly connected to the address output wire of WISHBONE bus.

5.3. Configuration Controller

The configuration controller shown in Figure 3 is implemented utilizing the basic gates. It plays a vital role in instantiating the asynchronous FIFOs as per the IP core and router requirements. This module issues the select signal to the multiplexer (not shown in the figure) to enable, disable/enable one module with respect to the router and IP control signals.

The configuration controller performs a run-time adaptive configuration of FIFOs as per router and PE requirements of NoC. Even though FIFO2 signals are active outside of the configuration controller, the configuration

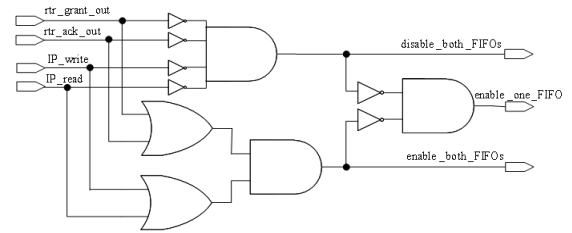


Figure 3. Run-time configuration controller

controller utilizes the asynchronous FIFO1 and utilizes the FIFO1 inputs as per the FIFO2 input output requirement.

5.4. Router to NI wrapper

The router to NI wrapper translates the configuration controller signals to router compatible signals. The input and output port of router consists of “ack” and “grant” signals. At the input port the adjacent router/local NI port requesting a transfer by asserting “ack_in” to HIGH, the

router asserts the “grant_out” to HIGH, if router is ready to receive data. Similarly, the output port of the router asserts “ack_out” to HIGH, when a router initiates a data transfer from router to processing core.

6. RESULTS AND COMPARISON

The run time configurable low power NI is synthesized by Xilinx ISE Complete Design suit 14.1 targeting Virtex-5 XC5VLX110T, 65nm copper CMOS process technology nominal corner at 1.0V and 85°C. The runtime instantiation of asynchronous FIFO offers a major impact on the performance of the proposed NI. The power analysis of the different runtime instantiation of the proposed NI is shown in Table.1. By setting the required constraint in the proposed design the IP core clock is attained at a maximum speed of 372 MHz and router side clock attained a maximum speed of 338 MHz irrespective of the configuration conditions of asynchronous FIFO. Therefore, the optimum worst case speed of the proposed design is 338 MHz.

The total power dissipation in any VLSI system is the summation of a switching power, a short circuit power and a leakage power. For an FPGA design with a minimum portion of device utilization, it is advisable to calculate the dynamic power (P_{dy}) dissipation instead of total power (PT). The power dissipation of proposed NI is calculated a nominal clock speed of 200 MHz with 50% toggle rate using the Xilinx XPower Estimator. The results show that the proposed NI offers a power improvement of 37%, when both FIFOs are not instantiated and 32%, when only one FIFO is instantiated as shown in Table 1 and the comparison with the existing work is shown in Table 2.

TABLE I. POWER REDUCTION OF PROPOSED DESIGN

FIFO Conditions	Power P _{dy} mW	Power Reduction %	Speed in MHz
2 FIFOs are inactive	45.19	37	338
1 FIFO is active	48.89	32	338
Both FIFOs are active	72.30	--	338

TABLE II. COMPARISON WITH EXISTING NI

S.No	Works	% of Power improvement	Latency cycles	Throughput MFlits/s
1	This work	37 / 32	2/3	169/112
2	AHB [8]	Constant	SNI-3/3/3 MNI-6/4/3	103/103/103 43/65/87
3	OCF [10]	Constant	SNA-3/3 MNA-10/4	154/110 118/123

The runtime instantiation of FIFO modules offers a significant reduction in power of NI as shown in Table I. The low latency WISHBONE bus wrapper and the finite state machine (FSM) based packing/unpacking modules offer less latency and higher throughput compared to existing work.

7. CONCLUSION AND FUTURE WORK

In order to improve the power of NI for NoC, the runtime instantiation of FIFO is a feasible solution. The proposed work is best suited for any asynchronous interface application which requires low power, low latency and high throughput data transfer. Experimental results show that the proposed low power run time reconfigurable NI offers a power improvement of 37%, when both FIFOs are inactive and 32%, when only one FIFO is active. This work could be improved by applying dynamic partial reconfiguration technique.

REFERENCES

- [1] W. J. Dally and B. Towles, “Route packets not wires: On-chip interconnection networks,” in *Proceedings of the 38th Annual Design Automation Conference (DAC-38)*, pp. 684–689, 2001.
- [2] L. Benini and G.D. Micheli, “Networks on Chips: A New SoC Paradigm,” *IEEE Computer*, vol. 35, no. 1, pp. 70–78, Jan. 2002.
- [3] S. P. Singh et al., “Generic Network Interfaces for Plug and Play NoC Based Architecture,” *Lecture Notes in Computer Science Springer Reconfigurable Computing: Architectures and Applications*, pp. 287–298, 2006.
- [4] OpenCores WISHBONE Specification, “WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores,” rev. B.4, 2010, http://cdn.opencores.org/downloads/wbspec_b4.pdf.
- [5] C. Cummings and P. Alfke, “Simulation and synthesis techniques for asynchronous FIFO design with asynchronous pointer comparison,” in *SNUG*, 2002.
- [6] M. Fattah, A. Manian, A. Rahimi, S. Mohammadi, “A High Throughput Low Power FIFO used for GALS NoC Buffers,” in *Proc. of IEEE Annual Symposium on VLSI (ISVLSI)*, pp.333–338, 2010.
- [7] E. Beigne and P. Vivet, “Design of On-chip and Off-chip Interfaces for a GALS NoC Architecture,” *ASYNC*, pp. 172–183, 2006.
- [8] M. Daneshmand et al., “Efficient network interface architecture for network-on-chips,” *NORCHIP*, pp 1–4, 2009.
- [9] M. Daneshmand et al., “Memory-Efficient On-Chip Network With Adaptive Interfaces,” *Computer-Aided Design of Integrated Circuits and Systems, IEEE Transactions on*, vol.31, no.1, pp.146–159, Jan. 2012.
- [10] B. Attia et al., “Design and implementation of network interface compatible OCP For packet based NOC,” *International Conference on Design & Technology of Integrated Systems in Nanoscale Era*, pp. 1–8, 2010.
- [11] B. Attia et al., “Network Interface Sharing for SoCs based NoC,” in *International Conference on Communications, Computing and Control Applications*, pp. 1–6, 2011.
- [12] K. Swaminathan, Lakshminarayanan G and Ko Seok-Bum, “High Speed Generic Network Interface for Network on Chip using Ping Pong Buffers,” in *International Symposium on Electronic System Design*, pp. 72–76, 2012.
- [13] D. Atienza et al., “Network-on-chip design and synthesis outlook,” *Integration of VLSI J* 41 (3), pp. 340–359, 2008.